Test report 18 – Sketch editor

Plan 17.7, step 4: drawing mode with grid in browser and Tauri · cad-eval (`sketch_edit.rs`, drawing state of sketches), cad-session (protocol `Sketch`), web UI (`app/sketch.ts`, `sketchUi.tsx`, `logic/sketchEditor.ts`, viewer) · 2026-10-24 · Branch worktree-agent-ab1441aa7672b7a32 · Commit 4abe40d · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 cores

Tools	cargo test -- workspace	clippy · fmt	Vitest · Playwright	Tauri T16	Snap + preview
9 + 2	**554 / 554**	**clean · clean**	**229 · 120 / 120**	**passed**	**4.0 ms**
line, 3 arcs, Bézier, rectangle, circle, round corner, select; close, delete	23 of them new	-D warnings	new: Z01–Z17	8 checks, native kernel	median per mouse move (browser, 42 segments)

1 Task and result

Plan 17.6: tools in the view, drawing in the sketch plane with a grid, snap with indicator, every action writes code into the `sketch` block at the cursor or creates one, undo as with dragging, editing via handles, deleting segments; the code remains the single source. Implemented:

- **Drawing mode** (toggle "Sketch", German UI: „Skizze“): camera perpendicular to the sketch plane – also under transformations, in `rotate_extrude` (xz plane) and in centered `linear_extrude` (bottom face); step grid, grid lines and grid points of the module or the sketch in the plane; toolbar, status line, Esc exits.
- **Tools** line (chain, a click on the start closes), arc through three points, arc around a center, tangent arc, Bézier (cubic), rectangle, circle, round corner (`corner(r)`), close, delete, select. Every action is a text edit in the existing language and one undo step.
- **Snap** to grid points and anchors (references such as `g[b, u]`, `k.ecke`), start, corners, end points, midpoints and intersections, arc midpoints, tangent (`line(length = …)`), horizontal/vertical, grid lines (`[g.b, 40]`), step size; Alt = free with step size. The marker shows the kind, the label shows the text that would be written.
- **Editing:** with "Select" the handles of all segments (existing handle logic: change a literal, rewrite a grid reference), a click on a segment sets the cursor, Del deletes.
- **Logic in Rust:** sketch and insertion point, drawing state, snap, code, preview, deletion, corners, grid, plane and ray intersection in `cad-eval`/`cad-session`; the UI forwards mouse and keys and applies the response. The same via `WorkerTransport` (Wasm) and `TauriTransport` (native engine process).

2 Decisions

Question	Decision
Where the code goes	Sketch = the `sketch` statement that contains the cursor or starts on its line. Insertion after the statement on the cursor line (or the last one before it), in the innermost `hole` block; on the `sketch {` line at the end; empty blocks are expanded. Without a sketch: new `sketch { … }` block at the end of the context module or the file. After every action the cursor is placed after the new code.
Current point	From the evaluation: `SketchInfo::steps` holds every segment statement in evaluation order with the state after it (contour, start, point, direction; geometry before `corner()`) – also for loops and module calls in the block. Line, arcs and Bézier continue the open contour if their first point is its end; otherwise `start(p)`. A new contour in the middle of a continuing one goes after its end.
Tool state	The engine is stateless: the chosen points (position, text, kind) go out with every request and come back with the response. At most one request in flight, the last mouse position wins, clicks in order.

Undo	One transaction per action with `isolateHistory` (also `start` + first line together); the source goes to the engine immediately, which processes messages in order – the next request sees the new text. Ctrl+Z also works with focus in the view.
References instead of numbers	Grid points and anchors write their reference; a rectangle from two grid points also gets the other corners as grid points. Free points: numbers rounded to the step size; exact points (end points, intersections …): shortest number with 6 digits. 2D points write `g[a, b]` (also with handle snap, instead of `g[a, b, _]`).
Plane	`sketch_frame`: node coordinate system, in `rotate_extrude` the xz plane, in centered `linear_extrude` the bottom face – for editor, handles (`Handle.frame`) and rendering (`OpInfo::Sketch.plane`).
Handles and tools	Drawing tools also draw on existing points (no handles there to catch the click); handles of all segments with "Select".
Tolerance	Snap radius in pixels (settings, 14 px) times pixel size at the ray hit point; the UI sends only ray, pixel size and radius.

3 Flow

```
Mouse moved     → Sketch { cursor, tool, points, ray, pixel, snap_px, free }        (≤ 1 in flight)
                 ← Sketch { plane, snap {point, text, kind, label}, guides, preview, prompt }
Click           → Sketch { …, click: true }
                 ← points (point stored)  or  edits + select (action done)
                   UI: one transaction (undo step), cursor after the code, SetSource immediately
Close/Del       → Sketch { tool: close | delete, cursor }  ← edits

Engine (cad-eval::sketch_edit, Session):  place → sketch, insertion point, drawing state, grid/anchors
                                          snap → point + text + kind     click → code   preview → polylines
UI (app/sketch.ts, viewer):               ray, keys, apply response, draw, camera
```

4 Tools

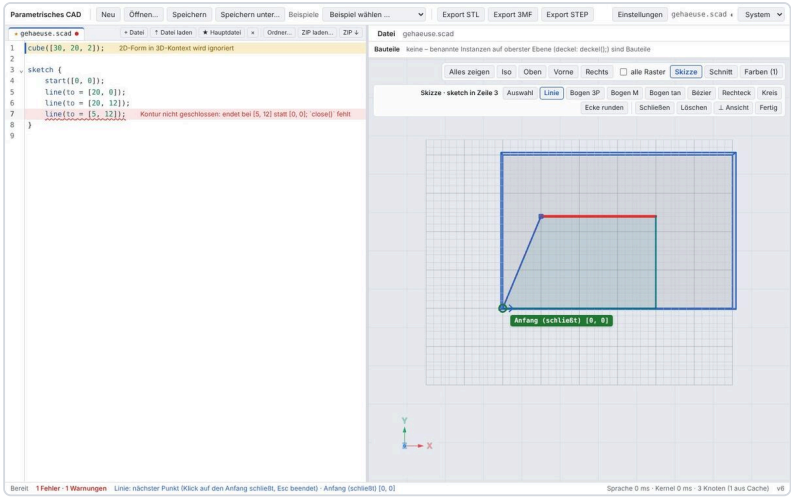
Each scenario draws with the mouse (Playwright, real pointer events) and checks the written code verbatim; left the code after the action, right the view.

Z02 Line

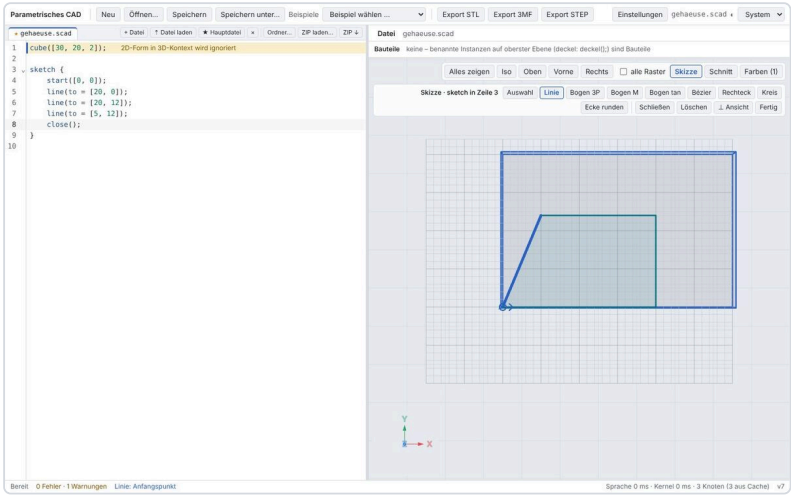
Without a sketch block at the cursor, the first action creates one (at the end of the file). Every click is an action: `start(…)` and the first line, then `line(to = …)` per point; a click on the start of the contour writes `close();`. Free points are rounded to the step size.

```
cube([30, 20, 2]);

sketch {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 12]);
    line(to = [5, 12]);
    close();
}
```



before the last click: snap "start (closes)" (German UI: „Anfang (schließt)“)



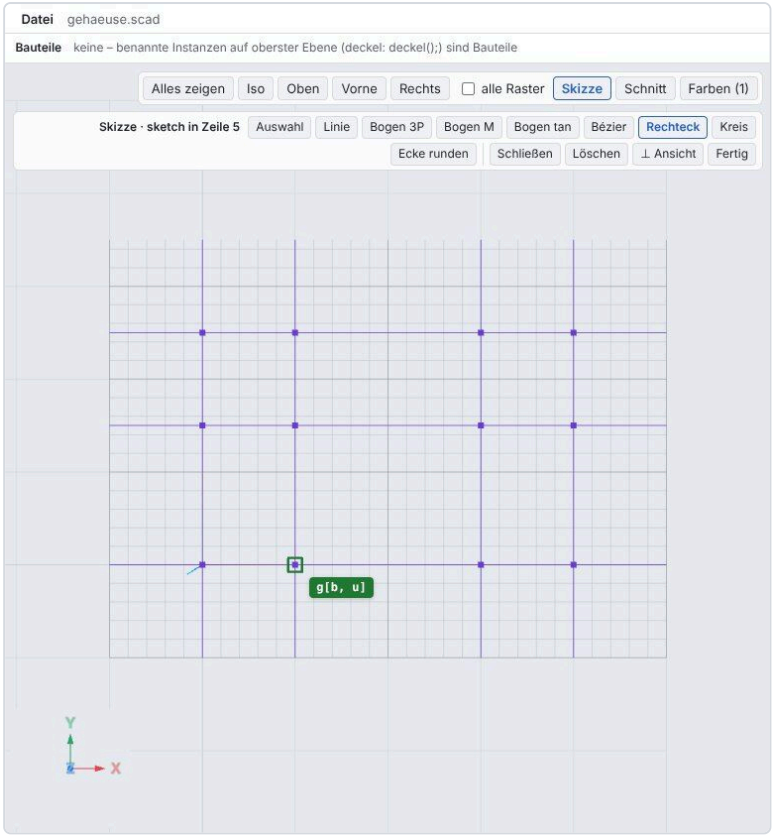
after the click: close(); closed contour

- new sketch block
- 4 actions
- close() by clicking on the start
- no errors

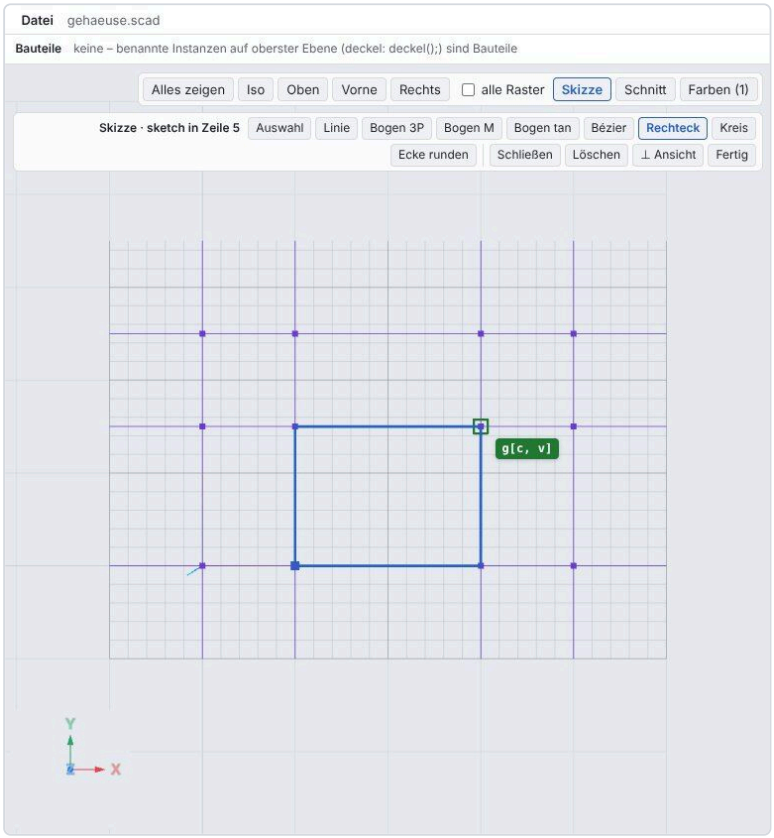
Z03 Rectangle on grid points

Snapping to grid points writes references. For a rectangle from two grid points, the other two corners also become grid points (`g[c, u]`, `g[b, v]`): if a grid level changes, the rectangle follows.

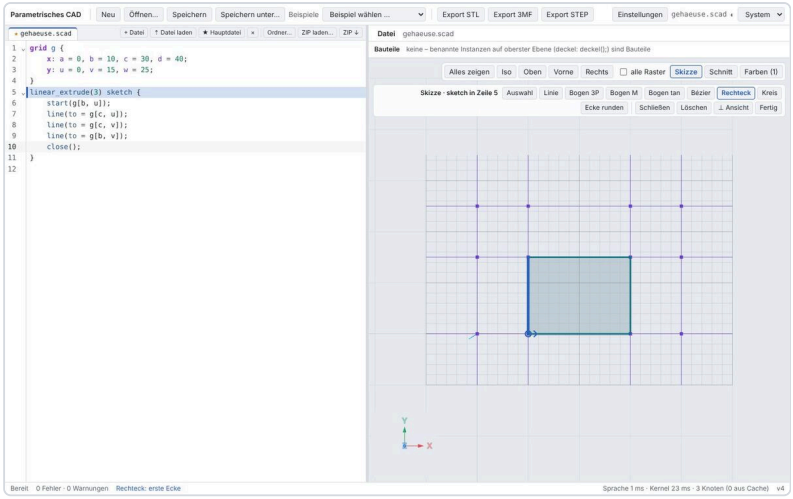
```
grid g {
    x: a = 0, b = 10, c = 30, d = 40;
    y: u = 0, v = 15, w = 25;
}
linear_extrude(3) sketch {
    start(g[b, u]);
    line(to = g[c, u]);
    line(to = g[c, v]);
    line(to = g[b, v]);
    close();
}
```



snap to grid point: square and text g[b, u]



second corner: preview of the rectangle



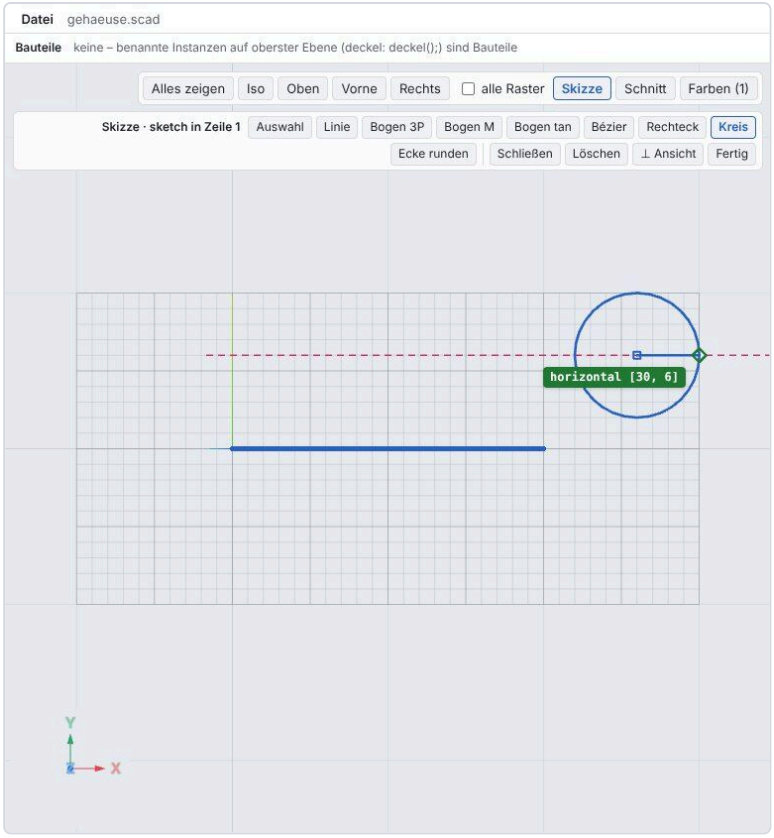
result: references instead of numbers

- snap text g[b, u] visible
- corners as g[…]
- preview 1 polyline

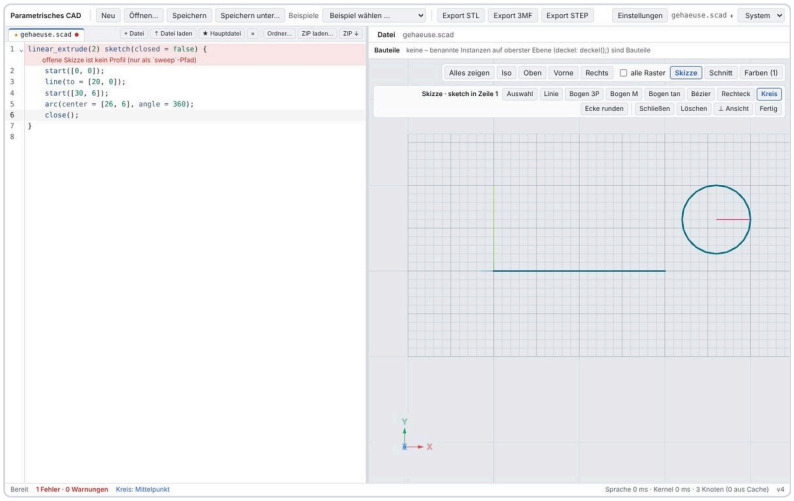
Z04 Circle

Center, then a point on the circle: `start(p); arc(center = c, angle = 360); close();` – a separate contour after the open one.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    start([30, 6]);
    arc(center = [26, 6], angle = 360);
    close();
}
```



preview: circle around the center



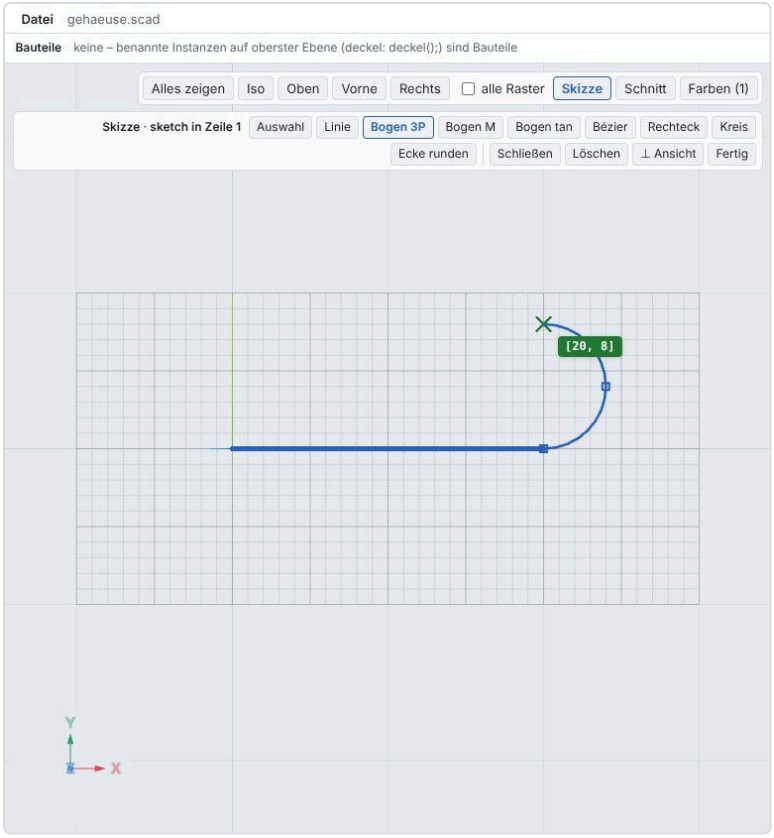
result

- start/arc(center, angle = 360)/close

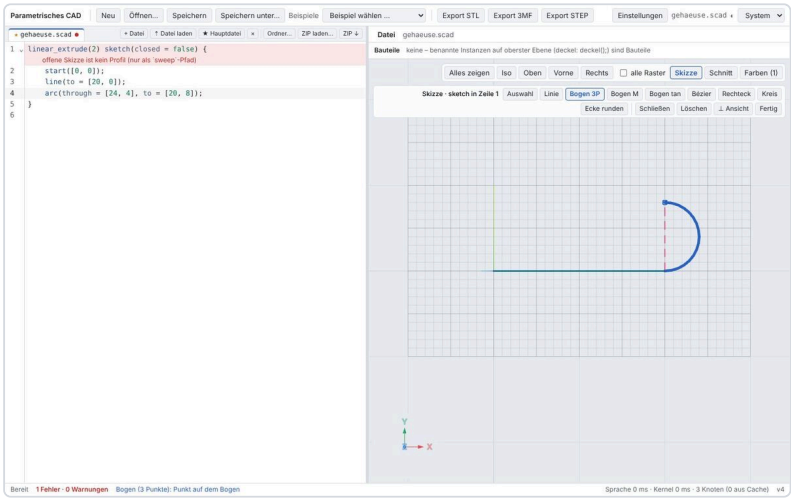
Z05 Arc through three points

Start (here the end of the contour, no new start), point on the arc, end point: `arc(through = q, to = p)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    arc(through = [24, 4], to = [20, 8]);
}
```



preview through start, through point and mouse



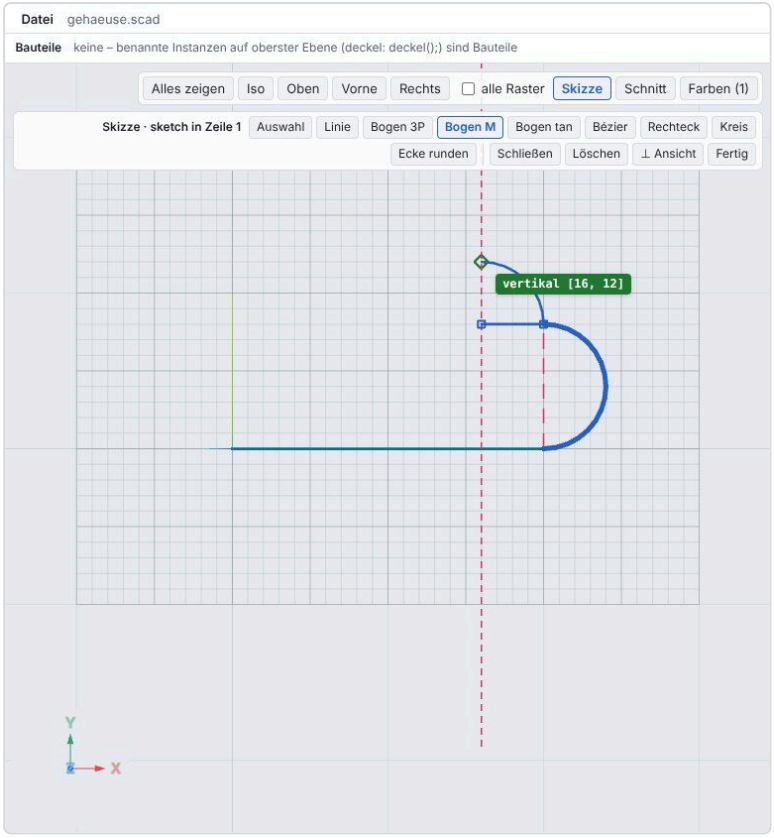
result

- arc(through = [24, 4], to = [20, 8])

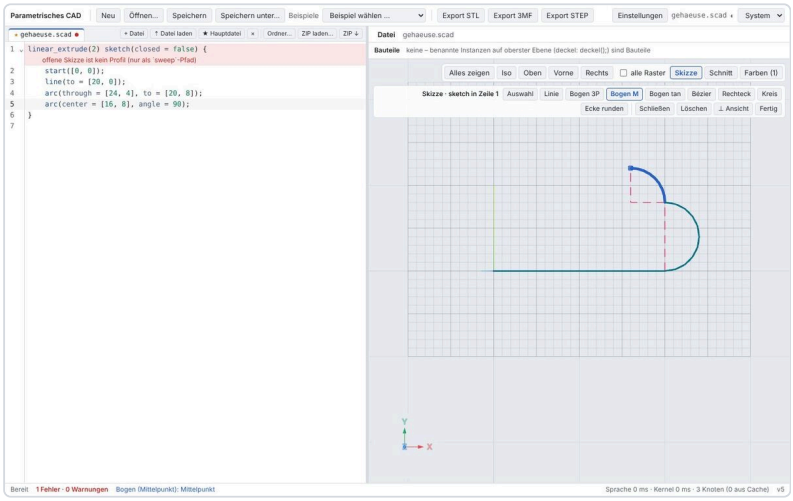
Z06 Arc around a center

Start, center, end angle: `arc(center = c, angle = a)`; the angle snaps to the angle step (15°), on a snapped point the result is `arc(center = c, to = p)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    arc(through = [24, 4], to = [20, 8]);
    arc(center = [16, 8], angle = 90);
}
```



preview with radius line



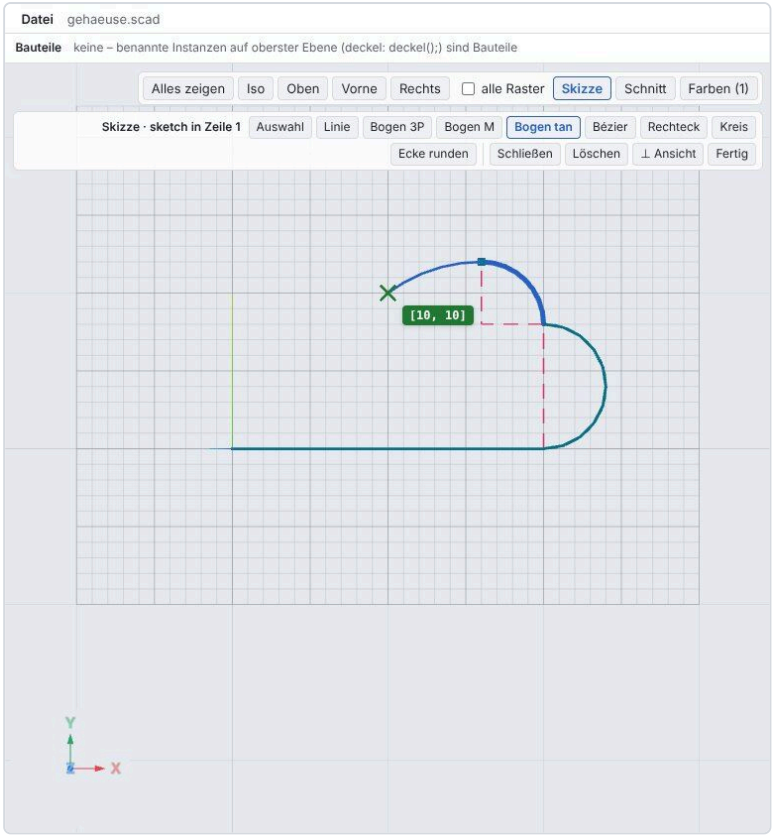
result

- arc(center = [16, 8], angle = 90)

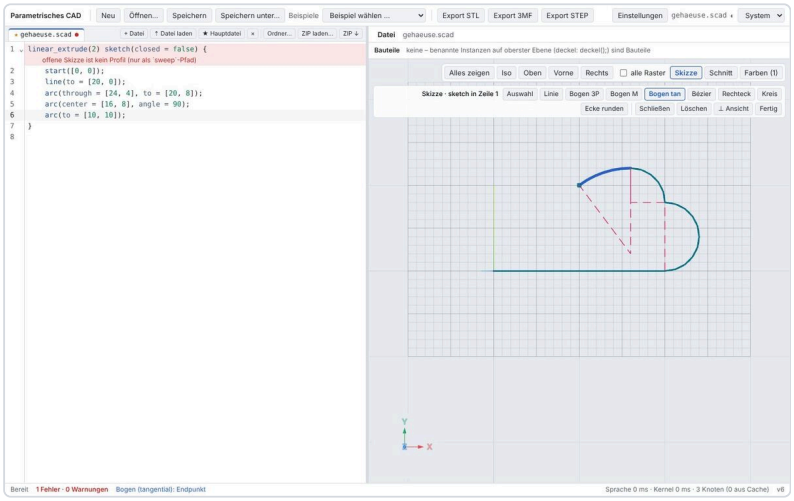
Z07 Tangent arc

One click: the arc continues the direction of the last segment, `arc(to = p)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    arc(through = [24, 4], to = [20, 8]);
    arc(center = [16, 8], angle = 90);
    arc(to = [10, 10]);
}
```



preview tangent at the end of the contour



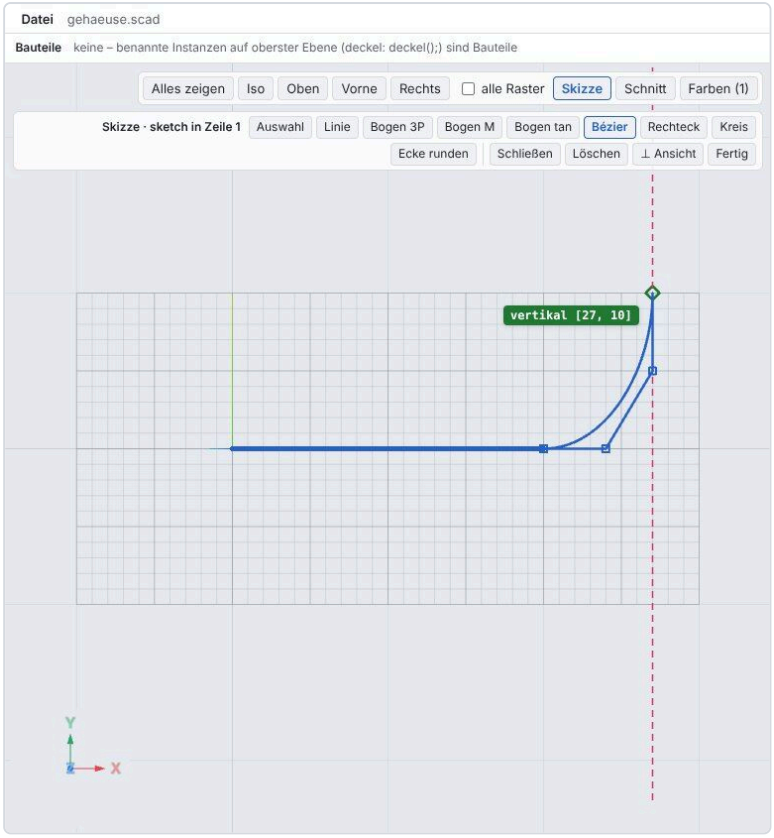
result

- arc(to = [10, 10])

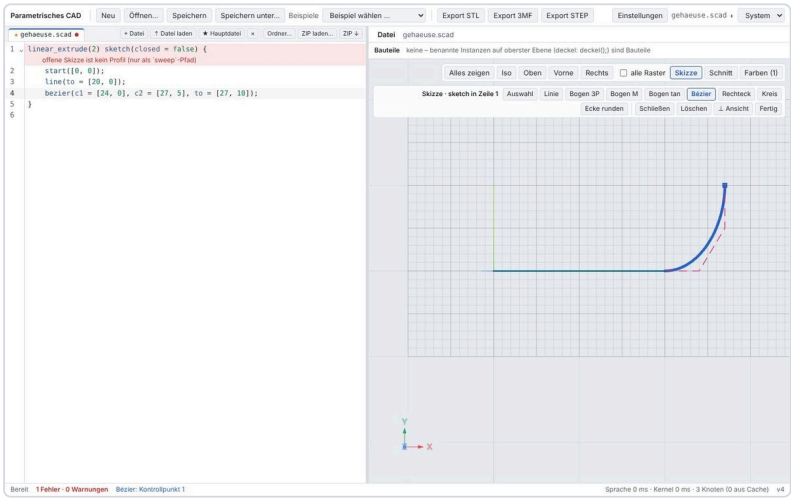
Z08 Bézier

Start, two control points, end: cubic curve `bezier(c1 = …, c2 = …, to = …)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    bezier(c1 = [24, 0], c2 = [27, 5], to =
[27, 10]);
}
```



preview with control polygon



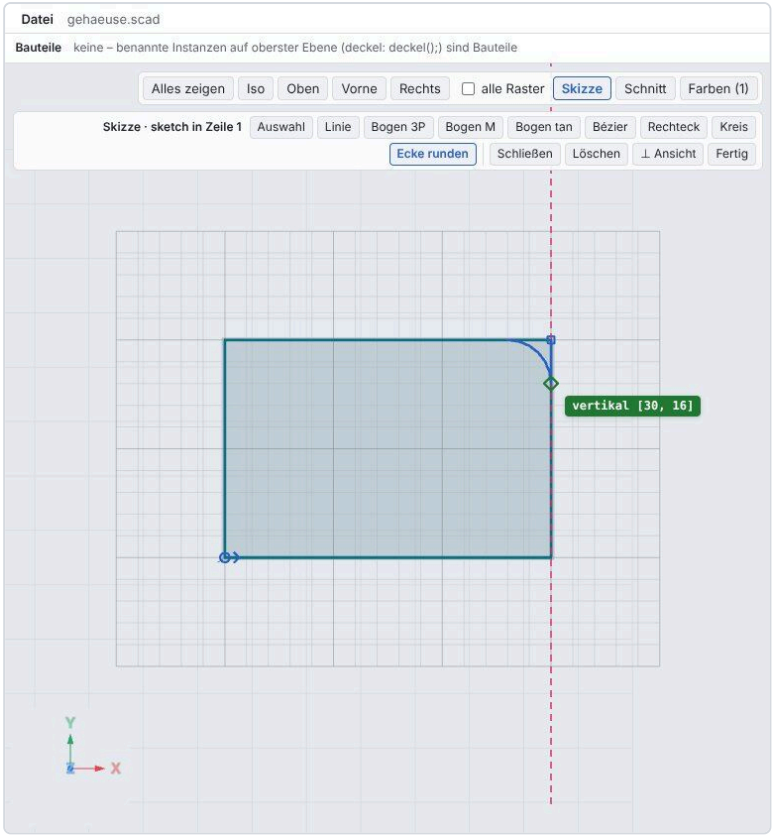
result

- bezier(c1 = [24, 0], c2 = [27, 5], to = [27, 10])

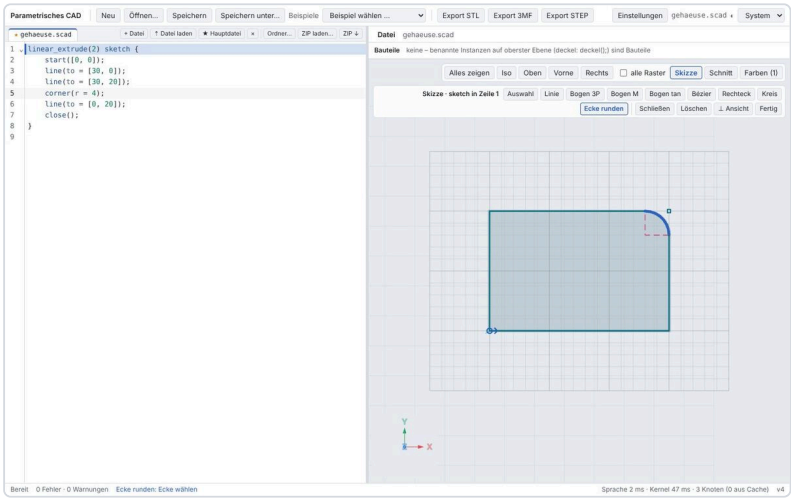
Z09 Round corner

Pick a corner (only corners between two segments snap), then the radius as the distance to the corner: `corner(r = …)` after the segment that ends there; on an already rounded corner the radius changes.

```
linear_extrude(2) sketch {
    start([0, 0]);
    line(to = [30, 0]);
    line(to = [30, 20]);
    corner(r = 4);
    line(to = [0, 20]);
    close();
}
```



corner picked, preview of the rounding



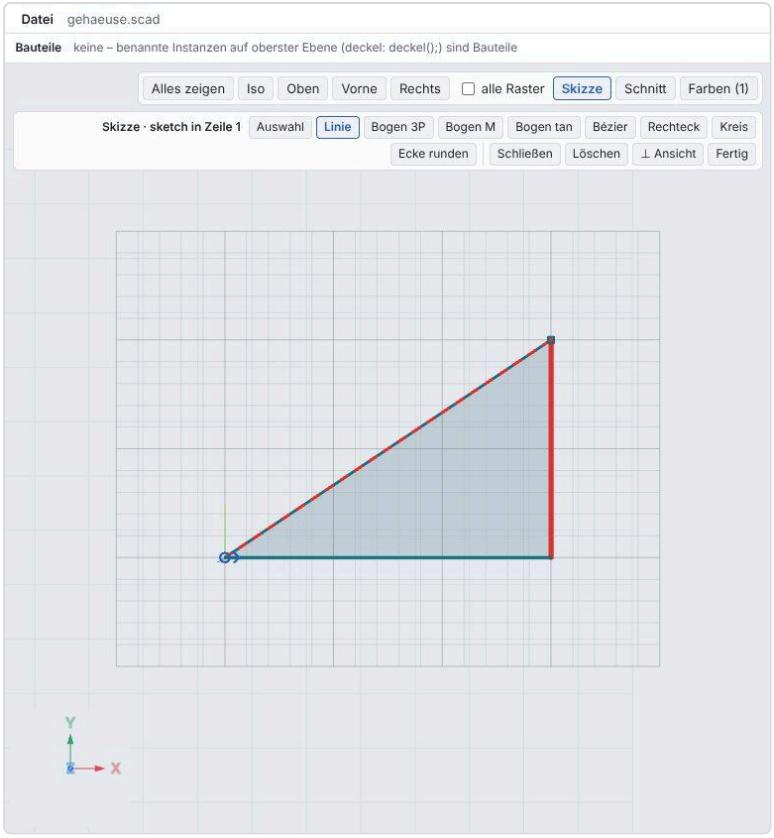
result

- corner(r = 4) after line(to = [30, 20])

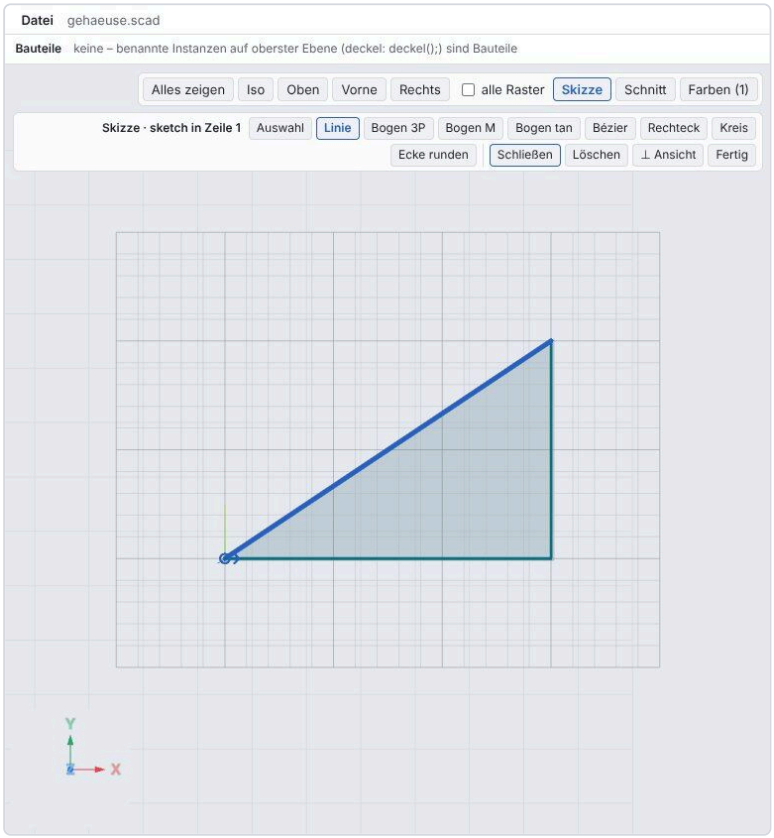
Z10 Close

"Close" writes `close();` for the open contour at the cursor; the error "contour not closed" (German: „Kontur nicht geschlossen“) disappears.

```
linear_extrude(2) sketch {
    start([0, 0]);
    line(to = [30, 0]);
    line(to = [30, 20]);
    close();
}
```



open contour (error: not closed)



after "Close": close();

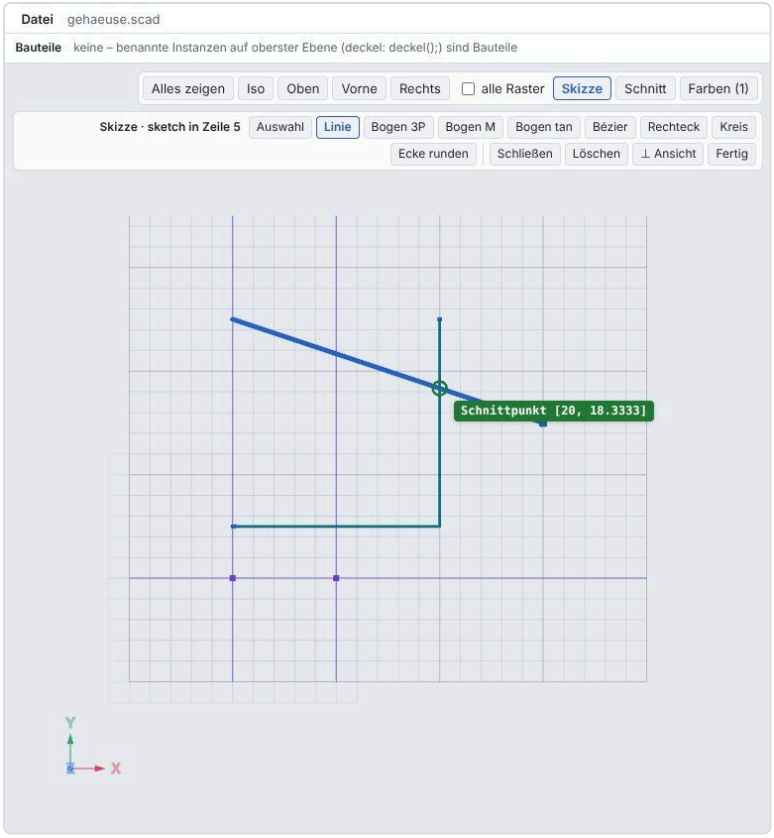
- close(); inserted
- no errors

5 Snap

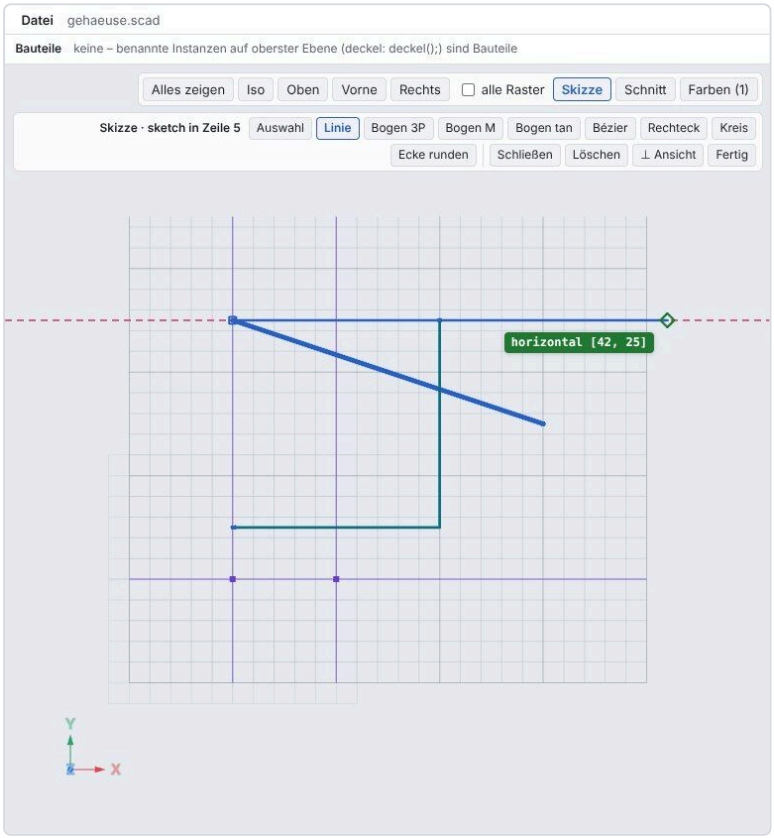
Z11 Snap kinds

Order: grid points and anchors (reference), start, corners, end points, midpoints, intersections and arc midpoints (numbers); otherwise tangent (writes `line(length = …)`), horizontal/vertical to the previous point and grid lines (`[g.b, 40]`), otherwise the step size. Alt: free with step size. The marker shows the kind (ring point, square grid, diamond direction, cross step).

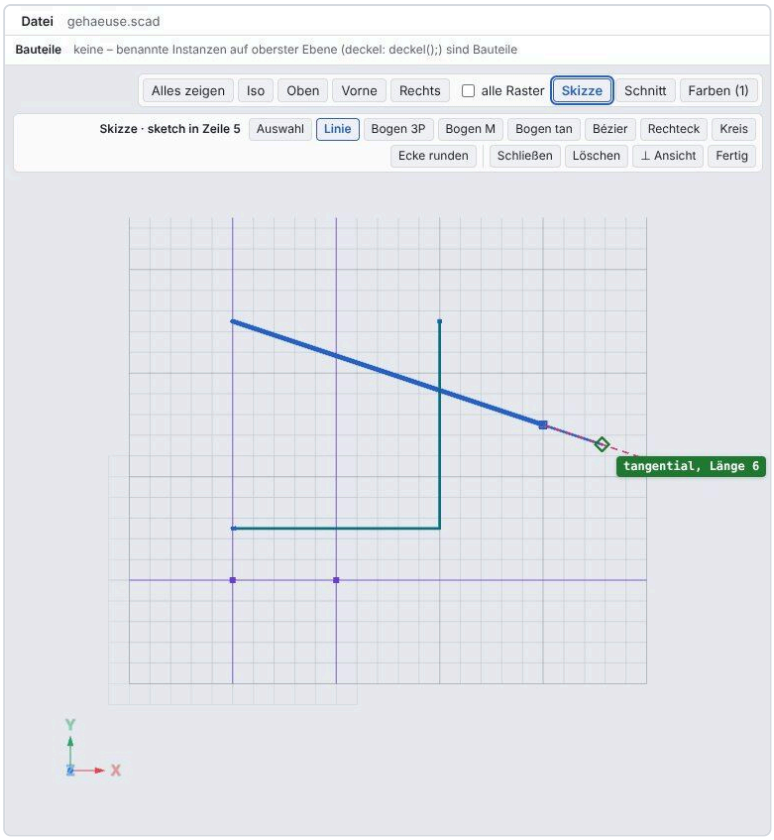
```
grid g {
    x: a = 0, b = 10;
    y: u = 0;
}
linear_extrude(2) sketch(closed = false) {
    start([0, 5]);
    line(to = [20, 5]);
    line(to = [20, 25]);
    start([0, 25]);
    line(to = [30, 15]);
    line(length = 6);
}
```



intersection of two lines (ring)



horizontal to the start point (diamond, guide line)



tangent to the last segment

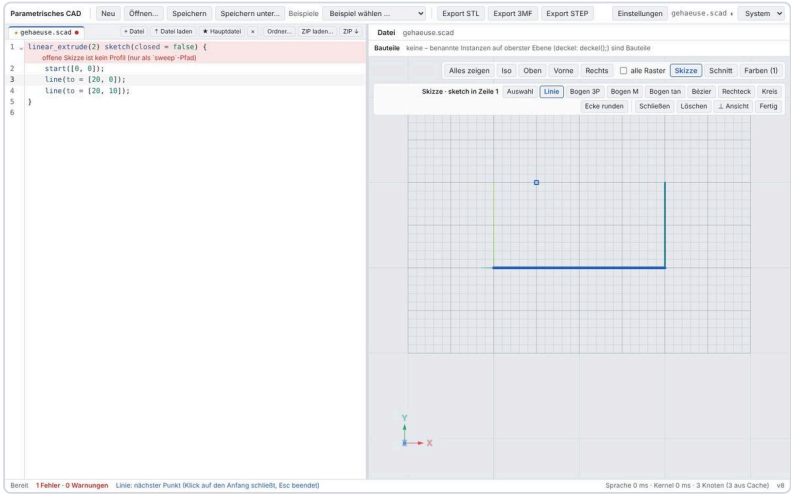
- end point → end: [20, 5]
- midpoint → mid: [10, 5]
- intersection → intersection: [20, 18.3333]
- grid point → grid: g[b, u]
- horizontal → horizontal: [42, 25]
- grid line → grid_line: [g.b, -6]
- Alt → step: [10, 0]
- tangent → tangent: [35.6921, 13.1026]

6 Editing: undo, handles, delete

Z12 Undo

Every action is a separate history step (also `start` + first line together); Ctrl+Z/Ctrl+Shift+Z also work with focus in the view.

```
// after two actions
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 10]);
    line(to = [5, 10]);
}
// Ctrl+Z, Ctrl+Z, Ctrl+Shift+Z
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 10]);
}
```



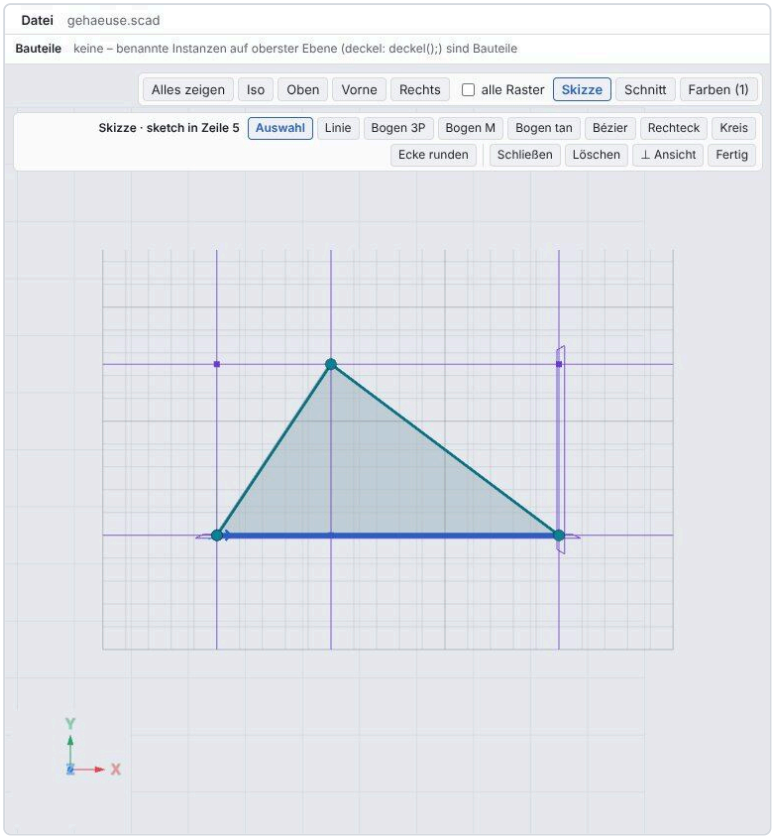
after undo, undo, redo

- undo reverts exactly one action
- redo restores it

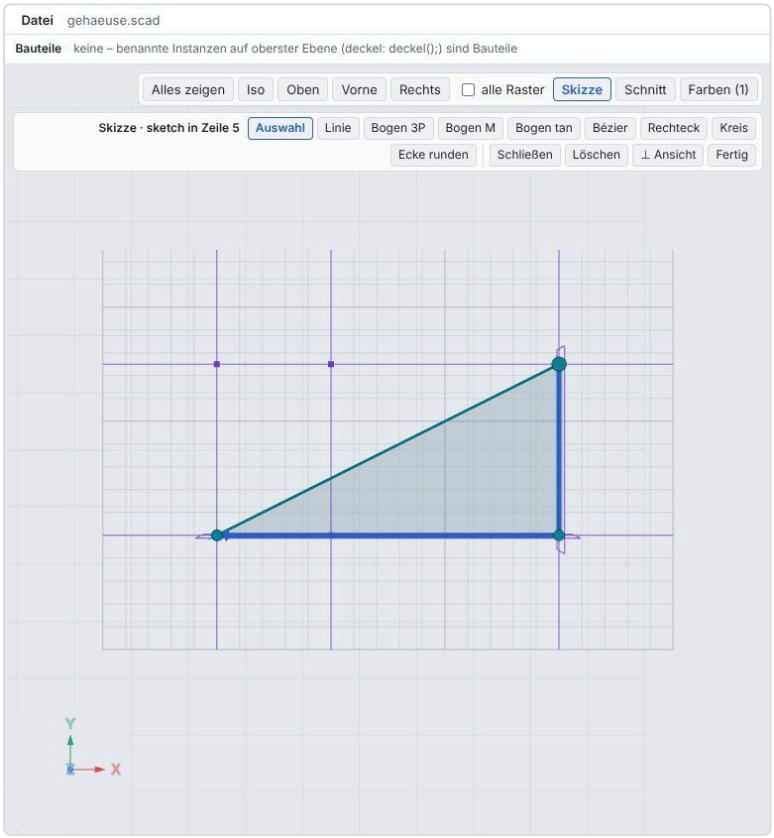
Z13 Dragging a handle

In drawing mode the view shows the handles of all segments of the sketch. Dragging uses the existing handle logic: on a grid point the reference is rewritten, literals are changed (Alt: numbers).

```
grid g {
    x: a = 0, b = 10, c = 30;
    y: u = 0, v = 15;
}
linear_extrude(2) sketch {
    start(g[a, u]);
    line(to = g[c, u]);
    line(to = g[c, v]);
    close();
}
```



handles of all segments (drawing mode)



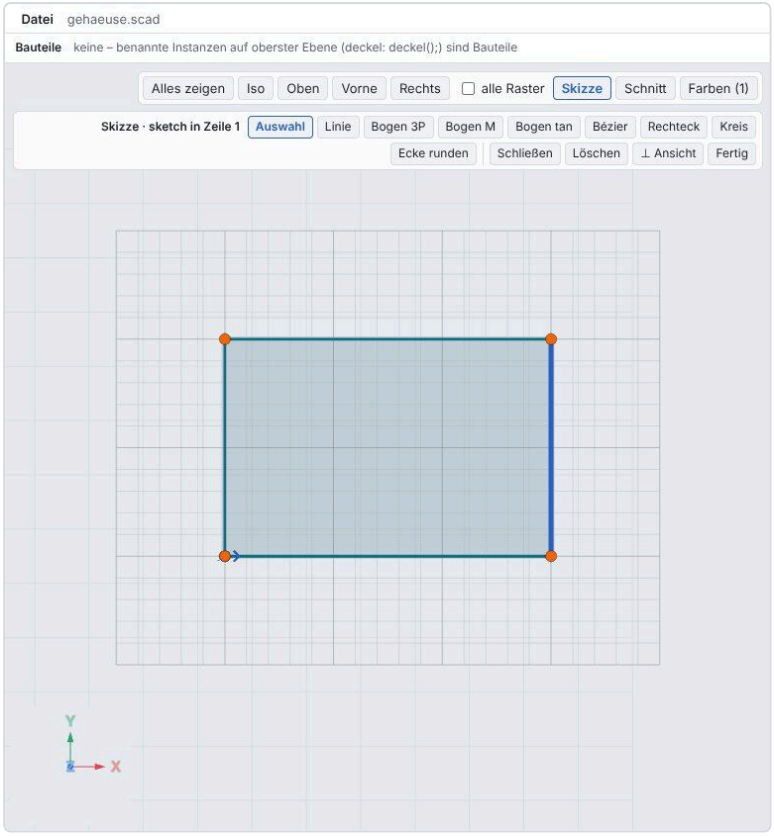
point dragged onto g[c, v]

- g[b, v] → g[c, v]

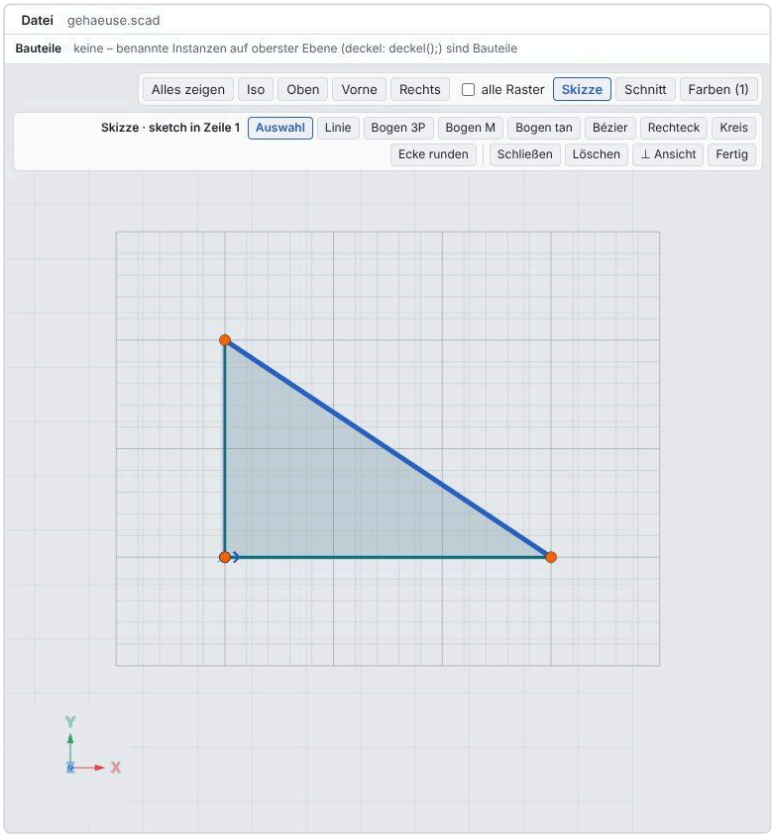
Z14 Deleting a segment

With the Select tool, a click on a segment puts the cursor on its line, Del (or "Delete") removes the statement. The only segment of a contour takes the contour with it, as does a `corner()` directly after it.

```
linear_extrude(2) sketch {
    start([0, 0]);
    line(to = [30, 0]);
    line(to = [0, 20]);
    close();
}
```



Select: segment under the mouse, click sets the cursor



after Del: line removed, the successor starts at the previous point

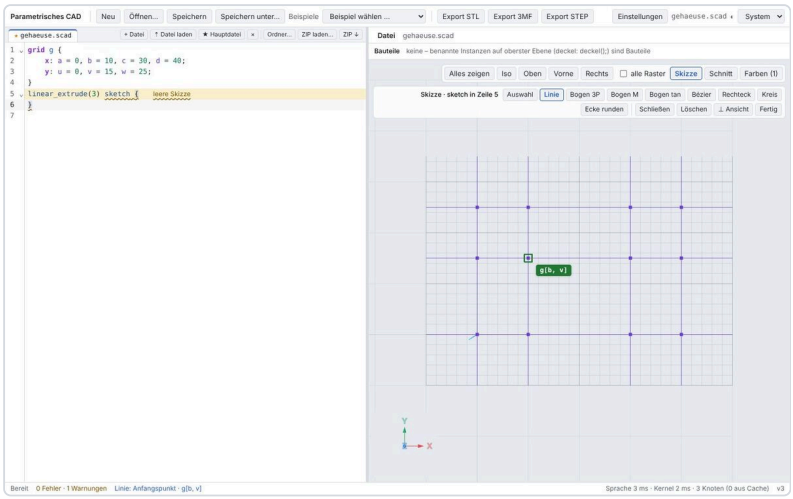
- line(to = [30, 20]) removed

7 Drawing mode, planes, theme

Z01 Drawing mode on/off

"Sketch" in the view toolbar turns drawing mode on: the camera looks perpendicular onto the plane of the sketch at the cursor, grid g and the step size (1 mm, coarser when zooming out) lie in the plane, the status line says what the tool needs next. Esc exits the mode.

```
grid g {
    x: a = 0, b = 10, c = 30, d = 40;
    y: u = 0, v = 15, w = 25;
}
linear_extrude(3) sketch {
}
```



drawing mode: toolbar, grid in the sketch plane, view from above

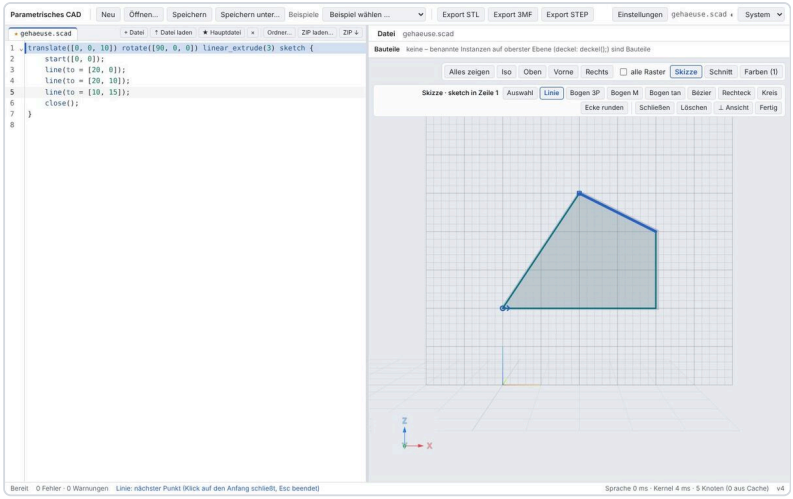
- view direction z = -1.00000
- grid lines 7, step grid 42
- Esc: mode off

Z15 Sketch under transformation, rotate_extrude

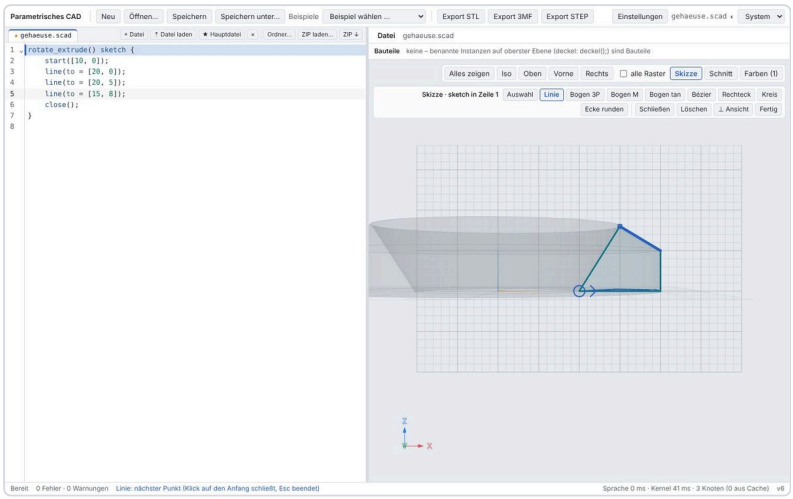
The sketch plane comes from the engine (`OpInfo.Sketch.plane`): under transformations its node coordinate system, in rotate_extrude the xz plane in which the profile lies in the body, in centered linear_extrude its bottom face. Camera, grid, snap and handles use the same plane; the values remain sketch coordinates.

```
translate([0, 0, 10]) rotate([90, 0, 0])
linear_extrude(3) sketch {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 10]);
    line(to = [10, 15]);
    close();
}

rotate_extrude() sketch {
    start([10, 0]);
    line(to = [20, 0]);
    line(to = [20, 5]);
    line(to = [15, 8]);
    close();
}
```



translate + rotate([90, 0, 0]): plane perpendicular to the y axis

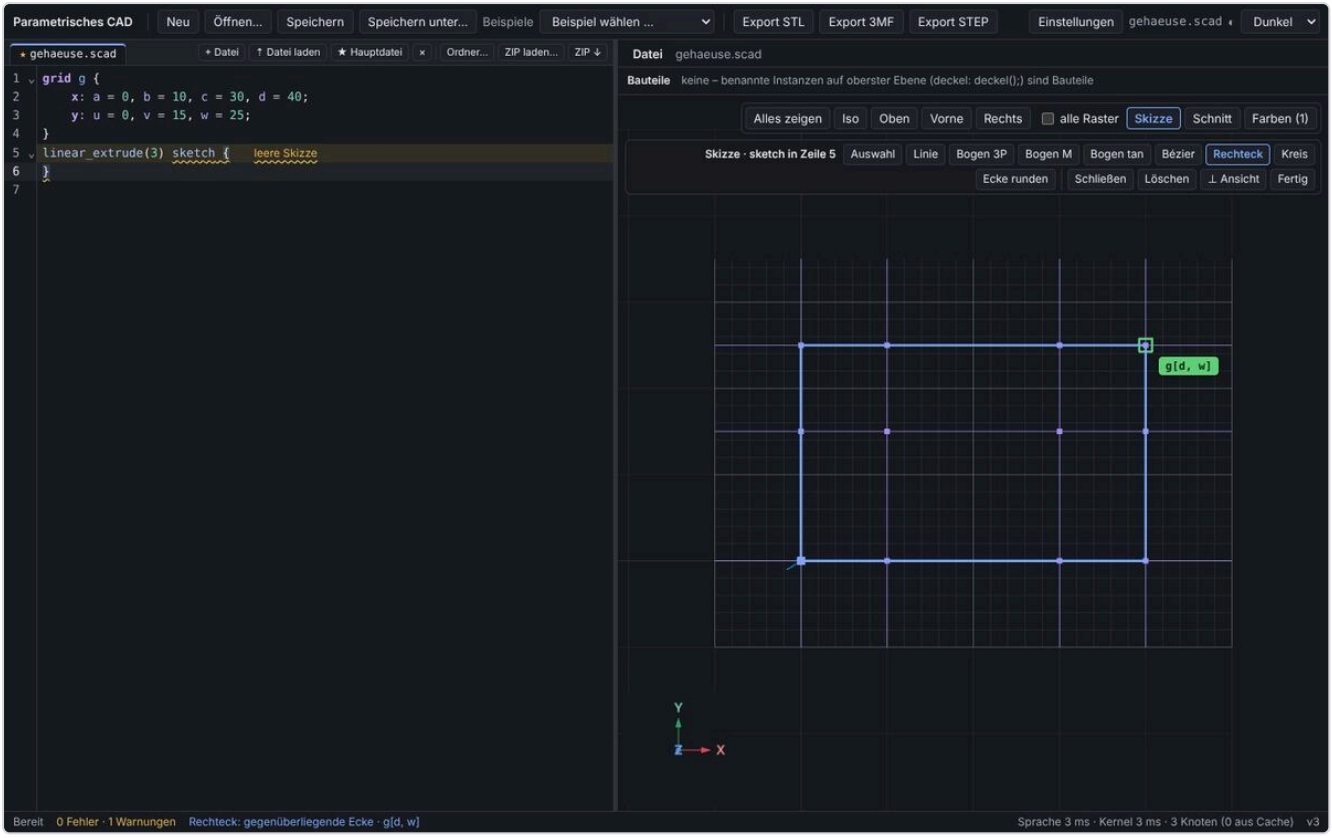


rotate_extrude: profile in the xz plane

- view y = 1.00000
- snap point z = 20 (world)
- rotate_extrude: view y = 1.00000

Z16 Dark theme

Grid, preview, snap marker and label use the theme colors (snap label as with handle snap).



drawing mode in the dark theme

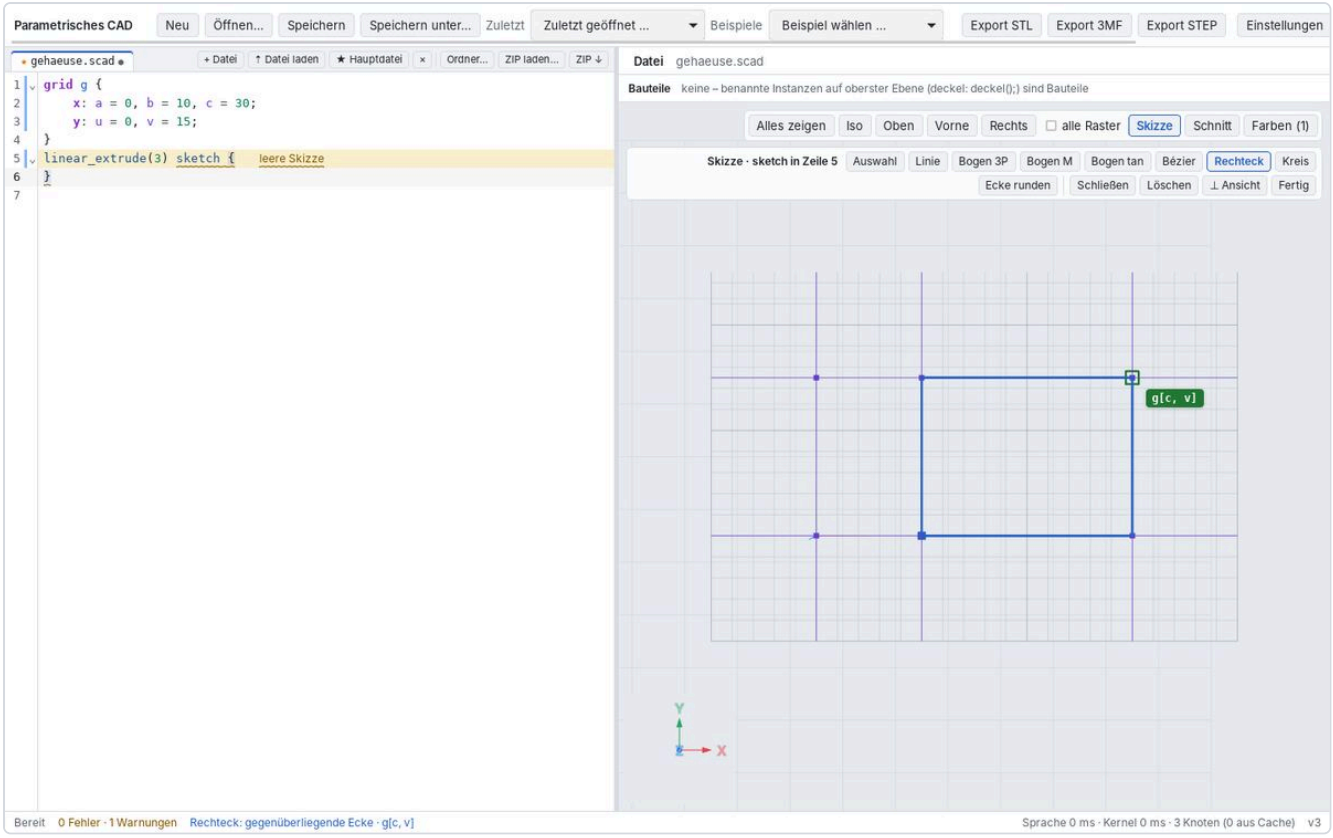
- contrast of the snap label 9.0:1

8 Desktop app (Tauri)

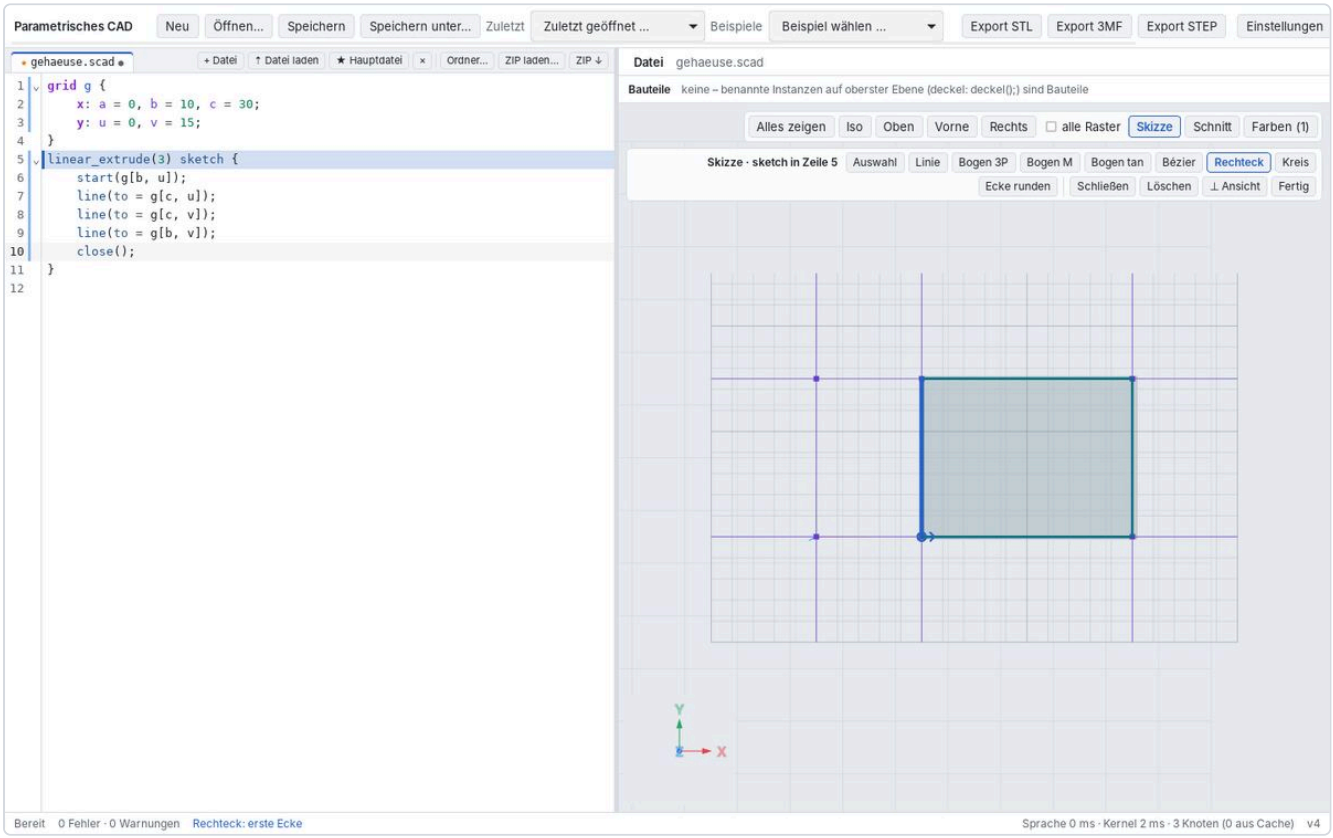
Scenario T16 in `web/e2e-tauri/desktop.e2e.mjs` (tauri-driver, WebKitWebDriver, Xvfb): the real app `cad` with the native engine process, messages via `TauriTransport`. The view is operated with pointer events on the canvas (WebDriver has no mouse over WebGL content that can be moved reliably).

Result: **passed** in 2.2 s, snap + preview median 13.0 ms per move.

- view perpendicular to the sketch plane (z = -1.00000)
- snap to grid point: g[b, u]
- snap label visible: g[b, u]
- preview of the rectangle
- code in the sketch block with grid references
- no errors, body computed natively
- undo reverts the action as one step
- drawing mode exited



rectangle: second corner g[c, v], preview and snap indicator

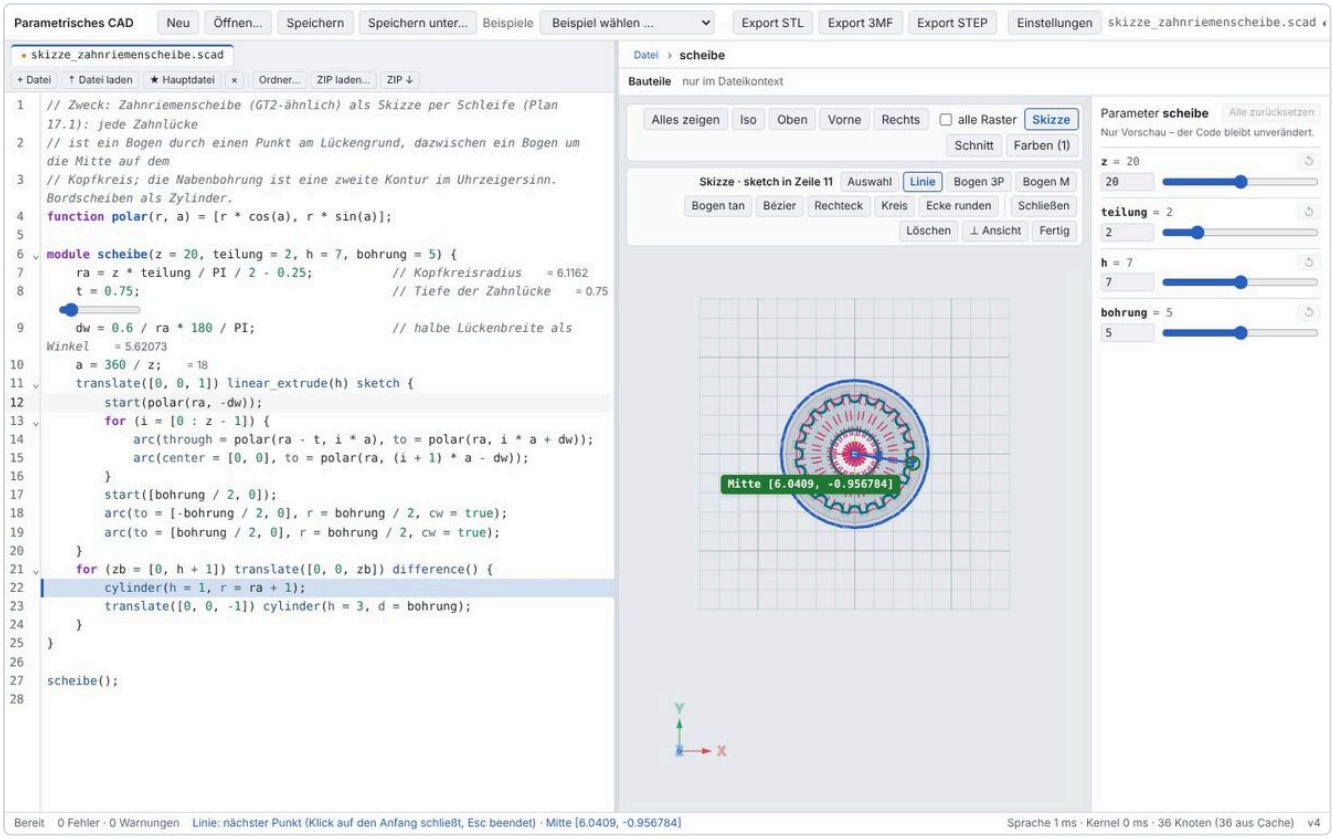


rectangle on grid points, code on the left

9 Measurements

Z17 Response times

Every mouse move sends the ray to the engine (worker, Wasm): snap against grid points, all end points, midpoints and intersections of the 42 segments, preview; at most one request in flight, the last position wins. Measured: request until response in the store.



timing belt pulley (sketch with for loop) in drawing mode

- 40 moves: median 4.0 ms, 95 % 20.0 ms, max 24.2 ms

10 Test runs

Run	Result
`cargo test --workspace -- --test-threads=2`	554 passed, 0 failed
`cargo clippy --workspace --all-targets -- -D warnings` · `cargo fmt --check`	clean · clean
`pnpm typecheck` · `pnpm test` (Vitest)	clean · 229 passed
`pnpm e2e` (Playwright, Chromium, full suite)	120 passed, 0 failed
Tauri E2E T16	passed

New Rust tests (23):

`sketch_edit::tests::step_rounding_writes_short_numbers` ok · `sketch_edit::tests::intersections_of_lines_and_arcs` ok · `sketch_edit::tests::sweep_continues_beyond_half_turn` ok · `anchors_are_snap_targets` ok · `arcs_bezier_circle` ok · `center_arc_to_a_snapped_point_keeps_the_reference` ok · `close_and_delete` ok · `dragging_a_sketch_point_onto_a_grid_point_keeps_the_short_form` ok · `corner_tool_inserts_and_updates` ok · `grid_points_write_references_and_rectangles_combine_them` ok · `line_between_statements_and_new_contour` ok · `hole_blocks_and_empty_blocks` ok · `new_sketch_at_the_end_of_the_file` ok · `new_sketch_in_the_context_module` ok · `line_continues_the_contour_at_the_cursor` ok · `plane_of_sketches_under_transformations` ok · `segment_under_the_mouse` ok · `snaps_by_kind` ok · `tangent_line_writes_length` ok · `sketch_op_info_carries_the_plane` ok · `sketch_under_a_transformation_snaps_in_its_plane` ok · `draws_lines_into_a_new_sketch_and_continues` ok · `syntax_errors_stop_drawing_and_select_finds_segments` ok

cad-eval `tests/sketch_edit.rs`: insertion point, new sketch in file and context module, chains, new contour after a continuing one, snap kinds, intersections, tangent, rectangle with grid references, arcs, Bézier, circle, center arc on grid points, inserting and changing corners, close, delete, `hole` and empty blocks, planes, anchors, segment under the mouse, handle snap in short form; unit tests in the module (arc angle over 180°, intersections, rounding). cad-session `tests/sketch_editor.rs`: protocol with ray (new sketch, chain, close, delete), plane under transformation, syntax errors, selection, `OpInfo.plane`. Vitest `logic/sketchEditor.test.ts`: grid, camera, markers, undo step with cursor.

11 Limitations

- Main file only: in another project file the mode does not start; sketches from included files are not a target.
- Statements in loops or modules are not deleted, and their corner does not get a `corner()` (message). The drawing state in the middle of a loop is the one after its last iteration.
- Snap only to the sketch, visible grids and anchors – not to body edges or other sketches; horizontal/vertical only relative to the previous point.
- Bézier only cubic, no spline tool; `line(by = …)` and `length` + `angle` are not generated (except tangent `line(length = …)`).
- A new sketch is a bare 2D sketch (to get a body: e.g. write `linear_extrude` in front of it).
- The camera keeps the orbit control's "z up": in rotated planes the sketch's y axis is not necessarily vertical on screen. With a drawing tool the left mouse button does not rotate the camera (right pans, wheel zooms, "Select" to rotate).
- Tauri: T16 operates the view with synthetic pointer events.